

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	S EZHILARASI	Reg. No.:	192521187
Section / Batch:	BTECH IT - B	Date:	13-08-2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both the GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I, S EZHILARASI (192521187) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ezhilarasi

CO3 AT1

SAST SIMUALTION

Name: S EZHILARASI

Register Number: 192521187

Course Code: CSA5803

Course Name: DevSecOps Engineering and Security Applications

SAST SIMULATION USING A REAL-LIFE SCENARIO

1. Title

Static Application Security Testing (SAST) Simulation for an Online Banking Application Using Synthetic Data

2. Real-Life Scenario

Scenario: Online Banking Web Application

Online banking applications are widely used by customers to perform activities such as logging in, checking account balances, transferring money, paying bills, and viewing transaction history.

For this SAST simulation, a **synthetic online banking web application** is considered. The application does not connect to any real bank, database, or payment system. All usernames, transactions, account numbers, source-code samples, vulnerabilities, and security results used in the simulation are artificially generated.

The objective is to simulate how a security team can identify application security weaknesses **before the application is deployed into production**.

Main Features of the Application

The simulated banking application contains the following modules:

1. User Login
2. Account Dashboard
3. Balance Checking
4. Money Transfer
5. Transaction History
6. User Profile
7. Logout

3. Problem Statement

A banking application handles sensitive information such as usernames, passwords, account details, and financial transactions. Security vulnerabilities in the application could allow attackers to access unauthorized information or manipulate transactions.

Therefore, Static Application Security Testing (SAST) is performed on the application's source code to identify potential security weaknesses without executing the application.

The simulation will:

- Generate synthetic source-code samples.
- Introduce intentional security vulnerabilities.
- Scan the source code using simulated SAST rules.

- Detect vulnerabilities.
- Classify vulnerabilities according to severity.
- Calculate a security risk score.
- Display the results using tables and charts.
- Provide recommended remediation actions.

4. Objective

The main objective is to simulate a SAST process for an online banking application using synthetic data.

Specific Objectives

- To understand the working principle of SAST.
- To identify security vulnerabilities in source code.
- To simulate automated source-code scanning.
- To classify vulnerabilities as Critical, High, Medium, or Low.
- To calculate an overall security score.
- To recommend suitable remediation techniques.
- To visualize the security scanning results.

5. SAST Simulation Workflow

The proposed workflow is:

Source Code → Preprocessing → Static Analysis → Rule Matching → Vulnerability Detection → Severity Classification → Risk Calculation → Report Generation → Remediation

Workflow Explanation

Step 1 – Source Code Collection

Synthetic source-code snippets representing different modules of the banking application are created.



Step 2 – Code Preprocessing

The source code is divided into functions and statements so that individual security rules can be applied.



Step 3 – Static Analysis

The simulator examines the code without executing it.



Step 4 – Security Rule Matching

Each statement is compared against predefined security rules.



Step 5 – Vulnerability Detection

If a statement matches a vulnerable pattern, the simulator generates a security finding.



Step 6 – Severity Classification

Each finding is assigned a severity:

- Critical
- High
- Medium
- Low



Step 7 – Risk Score Calculation

The simulator calculates an overall application security score.



Step 8 – Report Generation

The final results are displayed using tables, statistics, and charts.



Step 9 – Remediation

The simulator provides recommendations for fixing each vulnerability.

6. Synthetic Source-Code Dataset

The simulation uses artificial code snippets rather than real banking source code.

ID	Module	Synthetic Code Pattern	Vulnerability
S001	Login	`query = "SELECT * FROM users WHERE user=" + username + ""`	SQL Injection
S002	Login	`password = "admin123"``	Hardcoded Password
S003	Transfer	`eval(userInput)`	Code Injection
S004	Profile	`element.innerHTML = userName`	XSS
S005	Transfer	`http://bank.local/transfer`	Insecure HTTP
S006	Login	`console.log(password)`	Sensitive Data Exposure
S007	Database	`SELECT * FROM accounts`	Excessive Data Exposure
S008	Session	`sessionTimeout = 3600000`	Weak Session Configuration
S009	Transfer	`amount = request.amount`	Missing Input Validation
S010	Login	`token = "BANK_TOKEN_123"``	Hardcoded Secret

The dataset is completely synthetic and is used only for educational simulation.

7. Security Rules Used in the Simulation

The SAST simulator contains predefined rules.

Rule 1 – SQL Injection

Pattern:

SQL query + direct user input

Example:

```
query = "SELECT * FROM users WHERE user='" + username + "'"
```

Severity: Critical**Recommendation:** Use parameterized queries or prepared statements.**Rule 2 – Hardcoded Password****Pattern:**

```
password = "somePassword"
```

Severity: High**Recommendation:** Store credentials securely using environment variables or a secrets manager.**Rule 3 – Cross-Site Scripting****Pattern:**

```
innerHTML = userInput
```

Severity: High**Recommendation:** Validate and encode user-controlled input before displaying it.**Rule 4 – Code Injection****Pattern:**

```
eval(userInput)
```

Severity: Critical**Recommendation:** Avoid dynamic code execution and validate input using an allowlist.**Rule 5 – Insecure Communication****Pattern:**

```
http://
```

Severity: High**Recommendation:** Use HTTPS for all communication involving sensitive information.**Rule 6 – Sensitive Data Exposure****Pattern:**

```
console.log(password)
```

Severity: High**Recommendation:** Never log passwords, tokens, or confidential financial information.**Rule 7 – Missing Input Validation****Pattern:**

```
amount = request.amount
```

Severity: Medium

Recommendation: Validate data type, range, format, and business rules before processing.

Rule 8 – Hardcoded Secret

Pattern:

```
token = "BANK_TOKEN_123"
```

Severity: High

Recommendation: Store secrets outside the source code.

8. Simulation Steps

Step 1 – Define the Application

Create a conceptual online banking application containing Login, Dashboard, Transfer, Profile, and Transaction modules.

The application itself does not need to connect to a real banking system.

Step 2 – Generate Synthetic Source Code

Create artificial source-code snippets representing common programming mistakes.

For example:

```
let username = input.username;

let query =
  "SELECT * FROM users WHERE username='" + username + "'";
```

The simulator identifies the direct combination of a database query and user-controlled input.

Step 3 – Create SAST Rules

Create a set of security rules.

Example:

```
IF source contains "SELECT"
AND source contains user input
THEN
  Report SQL Injection
```

Another rule:

```
IF source contains "innerHTML"
AND source contains user input
THEN
  Report Cross-Site Scripting
```

Step 4 – Scan the Synthetic Code

The simulator scans each source-code snippet line by line.

For every line, it checks whether one or more predefined security patterns are present.

Example:

```
Scanning:
S001 → SQL Injection detected
S002 → Hardcoded Password detected
```

Step 5 – Generate Findings

Every detected vulnerability is stored as a finding.

Example:

Finding ID	Module	Vulnerability	Severity	Line
F001	Login	SQL Injection	Critical	12
F002	Login	Hardcoded Password	High	18
F003	Transfer	Code Injection	Critical	34
F004	Profile	XSS	High	41
F005	Transfer	Insecure HTTP	High	48

Step 6 – Assign Severity

The simulator assigns a severity score to every vulnerability.

Severity	Score
Critical	10
High	7
Medium	4
Low	1

The severity is determined based on the potential impact of the vulnerability.

For example:

- SQL Injection → Critical
- Code Injection → Critical
- Hardcoded Password → High
- XSS → High
- Missing Input Validation → Medium

9. Risk Score Calculation

A simple risk calculation can be used:

Risk Score = Severity Score × Occurrence

For example, if SQL Injection has a severity score of 10 and occurs twice:

Risk Score = 10 × 2 = 20

The total application risk can be calculated as:

Total Risk = Σ Individual Risk Scores

The simulator can then convert the risk into a security score.

Example:

Maximum Possible Risk = 100

Detected Risk = 42

Security Score = 100 - 42
= 58/100

Therefore:

Application Security Score = 58%

10. Risk Classification

The simulator classifies the application based on the calculated score.

Security Score	Security Status
90–100	Excellent
75–89	Good
50–74	Moderate
25–49	Poor
0–24	Critical

For the sample simulation:

Security Score = 58%

Therefore:

Security Status = MODERATE RISK

11. Sample SAST Scan Result

After scanning the synthetic source code, the simulator may produce the following result:

Severity	Number of Findings
Critical	2
High	5
Medium	2
Low	1
Total	**10**

Result Summary

```
Total Files Scanned      : 5
Total Lines Scanned      : 250
Total Vulnerabilities     : 10
Critical Vulnerabilities  : 2
High Vulnerabilities      : 5
Medium Vulnerabilities    : 2
Low Vulnerabilities       : 1
Security Score            : 58%
Risk Level                : MODERATE
```

12. Example Vulnerability Report

Finding F001 – SQL Injection

Module: Login

Severity: Critical

Detected Code:

```
query = "SELECT * FROM users WHERE username='" + username + "'";
```

Reason:

User input is directly concatenated into a database query.

Potential Impact:

An attacker could manipulate the database query and potentially access or modify unauthorized information.

Recommended Fix:

Use parameterized queries.

Finding F002 – Hardcoded Password

Module: Login

Severity: High

Detected Code:

```
password = "admin123";
```

Reason:

Sensitive credentials are stored directly in the source code.

Recommended Fix:

Use secure environment variables or a secrets management system.

Finding F003 – Cross-Site Scripting

Module: Profile

Severity: High

Detected Code:

```
element.innerHTML = userName;
```

Reason:

Untrusted user input is inserted directly into HTML.

Recommended Fix:

Validate and encode user input before displaying it.

13. Simulation of Remediation

After vulnerabilities are detected, the simulator can provide a "Fix Vulnerability" demonstration.

Before Fix

```
query = "SELECT * FROM users WHERE username='" + username + "'";
```

After Fix

```
query = "SELECT * FROM users WHERE username = ?";
```

The simulator changes the vulnerability status from:

OPEN → FIXED

The security score is recalculated after remediation.

14. Before and After Comparison

ParameterBefore RemediationAfter Remediation		
Critical Issues	2	0
High Issues	5	2
Medium Issues	2	1

Low Issues	1	1
Total Issues	10	4
Security Score	58%	86%
Risk Level	Moderate	Good

This demonstrates the effectiveness of fixing vulnerabilities identified by SAST.

15. Proposed Simulation Interface

The SAST simulation webpage can contain the following sections:

Dashboard

Display:

- Total Files Scanned
- Lines Scanned
- Total Vulnerabilities
- Critical Issues
- High Issues
- Medium Issues
- Security Score

Source Code Panel

Display the synthetic source code being scanned.

Scan Controls

Buttons:

```
[ Start Scan ]
[ Pause ]
[ Next Finding ]
[ Auto Scan ]
[ Reset ]
```

Vulnerability Panel

Display:

```
Vulnerability: SQL Injection
Severity: Critical
Module: Login
Line: 12
Status: Open
```

Security Score

A circular or horizontal progress bar can display:

```
SECURITY SCORE

58 / 100
MODERATE RISK
```

Results Chart

A bar or doughnut chart can show:

Critical : ■ 2
High : ■■■■ 5
Medium : ■■ 2
Low : ■ 1

16. Complete Simulation Flow

```
START
↓
Define Online Banking Application
↓
Generate Synthetic Source Code
↓
Load SAST Security Rules
↓
Start Static Scan
↓
Analyze Source Code
↓
Match Security Patterns
↓
Vulnerability Found?
├── NO → Continue Scanning
└── YES
    ↓
    Create Finding
    ↓
    Assign Severity
    ↓
    Calculate Risk
    ↓
    Display Finding
    ↓
    Continue Scanning
    ↓
    Generate SAST Report
    ↓
    Calculate Security Score
    ↓
    Apply Remediation
    ↓
    Rescan Application
    ↓
    Compare Before vs After
    ↓
END
```

17. Expected Outcome

The SAST simulation is expected to demonstrate that static analysis can identify security weaknesses in source code before the application is deployed.

The simulation should successfully:

8. Scan synthetic source-code data.
9. Detect predefined security vulnerabilities.
10. Classify vulnerabilities according to severity.
11. Calculate the application's security risk.
12. Display vulnerabilities in a structured report.
13. Recommend remediation techniques.
14. Demonstrate improved security after fixing vulnerabilities.
15. Generate a final SAST security score.

18. Conclusion

This project demonstrates a **Static Application Security Testing (SAST) simulation for an online banking application using synthetic data**. The simulation follows the complete security-testing lifecycle, beginning with source-code collection and ending with vulnerability remediation and rescan.

By using predefined security rules, the simulator can identify vulnerabilities such as SQL Injection, Cross-Site Scripting, Code Injection, hardcoded credentials, insecure communication, and missing input validation.

The before-and-after comparison demonstrates how remediation reduces application risk and improves the overall security score.

Since all source code, user information, account details, and transaction data are synthetic, the simulation can safely demonstrate real-world SAST concepts without using actual banking data.

Final Result:

The proposed SAST simulator successfully models the process of identifying, classifying, scoring, and remediating application security vulnerabilities using synthetic data.